Project Overview and Objectives

Project Title

Knowledge Assistant

Overview

This project aims to develop an internal knowledge assistant to enhance information accessibility within the company. The system will leverage a RAG architecture to retrieve relevant documents and generate answers based on user queries, with RBAC ensuring that users only access documents permitted by their roles (e.g., department, hierarchy, or project involvement). It will be hosted on a cost-effective cloud instance, using Milvus for vector storage and local document storage for security and performance.

Objectives

- Build a RAG system that retrieves and generates responses from documents based on user permissions.
- Implement RBAC to enforce access controls tied to department, hierarchy, and project roles.
- Host the solution on a low-cost cloud instance to optimize expenses.
- Utilize Milvus as the vector database for efficient document retrieval.
- Store documents locally on the cloud instance for enhanced security and speed.
- Ensure the system is secure, compliant with company policies, and maintainable.

Requirements Specification

Functional Requirements

- **User Authentication**
    - Users must authenticate via the company's existing identity provider (e.g., Active Directory, OAuth).
    - Secure login to determine user identity and roles.
- **Role-Based Access Control (RBAC)**
    - Permissions assigned based on:

        - Department (e.g., IT, HR, Legal).
        - Hierarchy (e.g., staff, manager).
        - Projects (e.g., Project Alpha, Project Beta).
    - Higher-level users access lower-level general documents unless restricted.
- **Document Management**
    - Documents stored in folders where the last folder path segment indicates the permission group (e.g., Docs/HR for HR documents).
    - System parses folder paths to enforce access permissions.
- **Query Processing**
    - Users submit natural language queries.
    - System retrieves permitted documents and generates answers using a small language model (e.g., Deepseek-r1-1.5b).
- **Ticketing Integration**
    - If no answer is found, suggest opening a ticket in the company's ticketing system.
    - Allow users to edit ticket content and select a department.
- **User Interface**
    - Simple web interface for login, query submission, and response display.

Non-Functional Requirements

- **Performance**
    - Query responses within 5 seconds.
    - Efficient resource usage to suit a low-cost cloud instance.
- **Security**
    - Encrypt stored documents.
    - Enforce strict access controls.
    - Comply with company data protection policies.
- **Scalability**

    - Support current document and user volumes, with potential for growth.
- **Maintainability**
    - Use open-source tools with strong community support for ease of updates.

System Architecture Design

Architecture Overview

The system employs a microservices architecture with the following components:

- **Frontend**
    - Web app (Streamlit or Flask) for user interaction.
- **Authentication Service**
    - Integrates with the company's identity provider to verify users and fetch roles.
- **RAG Service**
    - Built with LangChain, processes queries, retrieves documents, and generates answers.
    - Filters documents based on RBAC permissions.
- **Vector Database**
    - Milvus stores document embeddings for fast similarity searches.
- **Language Model**
    - Deepseek-r1-1.5b, hosted locally via Ollama for answer generation.
- **Document Storage**
    - Local file system on the cloud instance, organized by permission-based folders.
- **Ticketing Integration**
    - API integration with the company's ticketing system.

Data Flow

- User logs in via the frontend.

- Authentication service verifies credentials and retrieves roles.
- User submits a query.
- RAG service:
    - Identifies accessible folders based on roles.
    - Queries Milvus for relevant documents within those folders.
    - Generates an answer using Deepseek-r1-1.5b.
- If no answer is found, prompts ticket creation with editable details.

Technology Stack Justification

Frontend

- **Streamlit or Flask**: Streamlit offers rapid prototyping for simple interfaces, while Flask provides customization flexibility. Both are lightweight and cost-effective, aligning with the low-cost cloud goal.

Authentication

- **OAuth 2.0 or SAML**: Standard protocols compatible with most identity providers, ensuring secure and seamless integration.

RAG Framework

- **LangChain**: An open-source framework ideal for RAG systems, simplifying retrieval and generation integration. Its extensive documentation and community support ease the learning curve.

Vector Database

- **Milvus**: Chosen for its high performance in vector similarity searches, scalability, and open-source nature. It's optimized for RAG applications, offering fast query times and low resource demands, despite requiring a learning path. This justifies its selection over simpler alternatives, given its long-term benefits for document retrieval efficiency.

Language Model

- **Deepseek-r1-1.5b**: A compact, efficient model runnable on modest hardware, minimizing costs. It's sufficient for internal use with a focused document set.

Document Storage

- **Local File System**: Storing documents locally on the cloud instance enhances security by keeping data in-house and reduces latency compared to external solutions (e.g., OneDrive), supporting your preference for local storage.

Ticketing Integration

- **REST API**: Leverages the company's existing ticketing system API for straightforward integration.

Reasoning: This stack balances cost, performance, and security. Milvus, though new to you, is justified by its superior vector search capabilities, critical for RAG efficiency. Local storage aligns with your choice for control and speed.

Security and Compliance Plan

Security Measures

- **Authentication**
    - Use secure protocols (e.g., OAuth 2.0, SAML).
    - Implement token-based API access.
- **Authorization**
    - Enforce RBAC rigorously.
    - Audit access logs periodically.
- **Data Encryption**
    - Encrypt documents at rest (e.g., AES-256).
    - Use HTTPS for data in transit.
- **Secure Hosting**
    - Configure cloud instance with firewalls and restricted access.
    - Apply regular security patches.

Compliance

- Adhere to company data protection policies (e.g., GDPR if applicable).

- Conduct security audits and penetration tests.

Testing and Quality Assurance Plan

Testing Phases

- **Unit Testing**
    - Validate individual components (e.g., RBAC, RAG service).
- **Integration Testing**
    - Ensure component interoperability (e.g., frontend to RAG).
- **User Acceptance Testing**
    - Verify usability with end-users.
- **Performance Testing**
    - Confirm query response times and resource efficiency.
- **Security Testing**
    - Identify vulnerabilities via penetration testing.

Quality Assurance

- Use CI/CD pipelines for automated testing.
- Conduct code reviews for quality and maintainability.

Deployment and Maintenance Plan

Deployment

- Package system in Docker containers for portability.
- Deploy on a low-cost cloud instance (e.g., DigitalOcean $5/month).
- Set up monitoring (e.g., Prometheus, Grafana) for performance tracking.

Maintenance

- Update system with patches and enhancements.
- Monitor resource usage to stay within budget.

- Provide staff training and documentation.
- Utilize open-source community support for troubleshooting.

Developer Brief

Project Title

Knowledge Assistant

Overview

Build an internal knowledge assistant using a RAG system with RBAC, hosted on a low-cost cloud instance. It will use Milvus for vector storage, Deepseek-r1-1.5b for generation, and local document storage.

Key Components

- **Frontend**: Streamlit or Flask web app.
- **Authentication**: Integrate with company identity provider.
- **RAG Service**: LangChain-based, with Milvus and Deepseek-r1-1.5b.
- **Document Storage**: Local file system with folder permissions.
- **Ticketing**: API integration for ticket creation.

Development Guidelines

- Adhere to the architecture and stack outlined.
- Prioritize RBAC security and system optimization.
- Document code thoroughly.
- Use CI/CD for testing and deployment.

Resources

- LangChain Documentation
- Milvus Documentation
- Ollama
- Streamlit Documentation
- Flask Documentation